Decision Tree Classification Algorithm

Introduction

- Decision Tree is a **Supervised learning technique** that can be used for both classification and Regression problems, but mostly it is preferred for solving Classification problems.

- It is a tree-structured classifier, where **internal nodes represent the features of a dataset, branches represent the decision rules** and **each leaf node represents the outcome.**

- In a Decision tree, there are two nodes, which are the **Decision Node** and **Leaf Node.** Decision nodes are used to make any decision and have multiple branches, whereas Leaf nodes are the output of those decisions and do not contain any further branches.

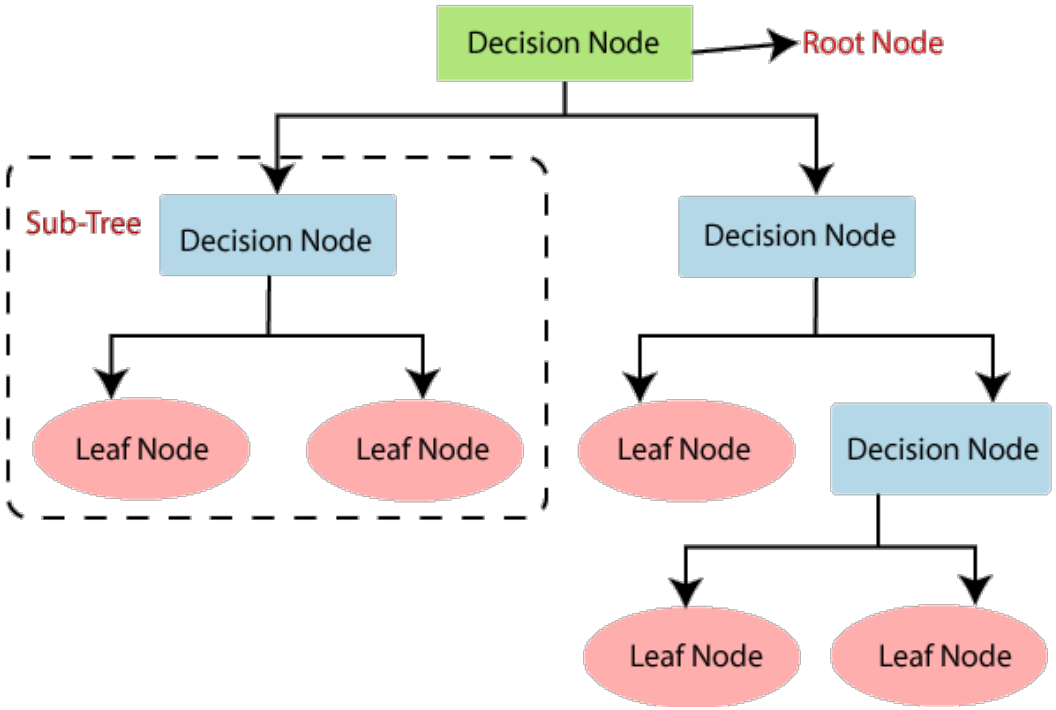
- The decisions or the test are performed on the basis of features of the given dataset.

- ***It is a graphical representation for getting all the possible solutions to a problem/decision based on given conditions.***

• It is called a decision tree because, similar to a tree, it starts with the root node, which expands on further branches and constructs a tree-like structure.

• In order to build a tree, we use the **CART algorithm,** which stands for **Classification and Regression Tree algorithm.**

• A decision tree simply asks a question, and based on the answer (Yes/No), it further split the tree into subtrees.



General structure of a decision tree

Why use Decision Trees?

- Decision Trees usually mimic human thinking ability while making a decision, so it is easy to understand.

- The logic behind the decision tree can be easily understood because it shows a tree-like structure.

Decision Tree Terminologies

- **Root Node:** Root node is from where the decision tree starts. It represents the entire dataset, which further gets divided into two or more homogeneous sets.
- **Leaf Node:** Leaf nodes are the final output node, and the tree cannot be segregated further after getting a leaf node.
- **Splitting:** Splitting is the process of dividing the decision node/root node into sub-nodes according to the given conditions.
- **Branch/Sub Tree:** A tree formed by splitting the tree.
- **Pruning:** Pruning is the process of removing the unwanted branches from the tree.
- **Parent/Child node:** The root node of the tree is called the parent node, and other nodes are called the child nodes.

How does the Decision Tree algorithm Work?

- In a decision tree, for predicting the class of the given dataset, the algorithm starts from the root node of the tree. This algorithm compares the values of root attribute with the record (real dataset) attribute and, based on the comparison, follows the branch and jumps to the next node.

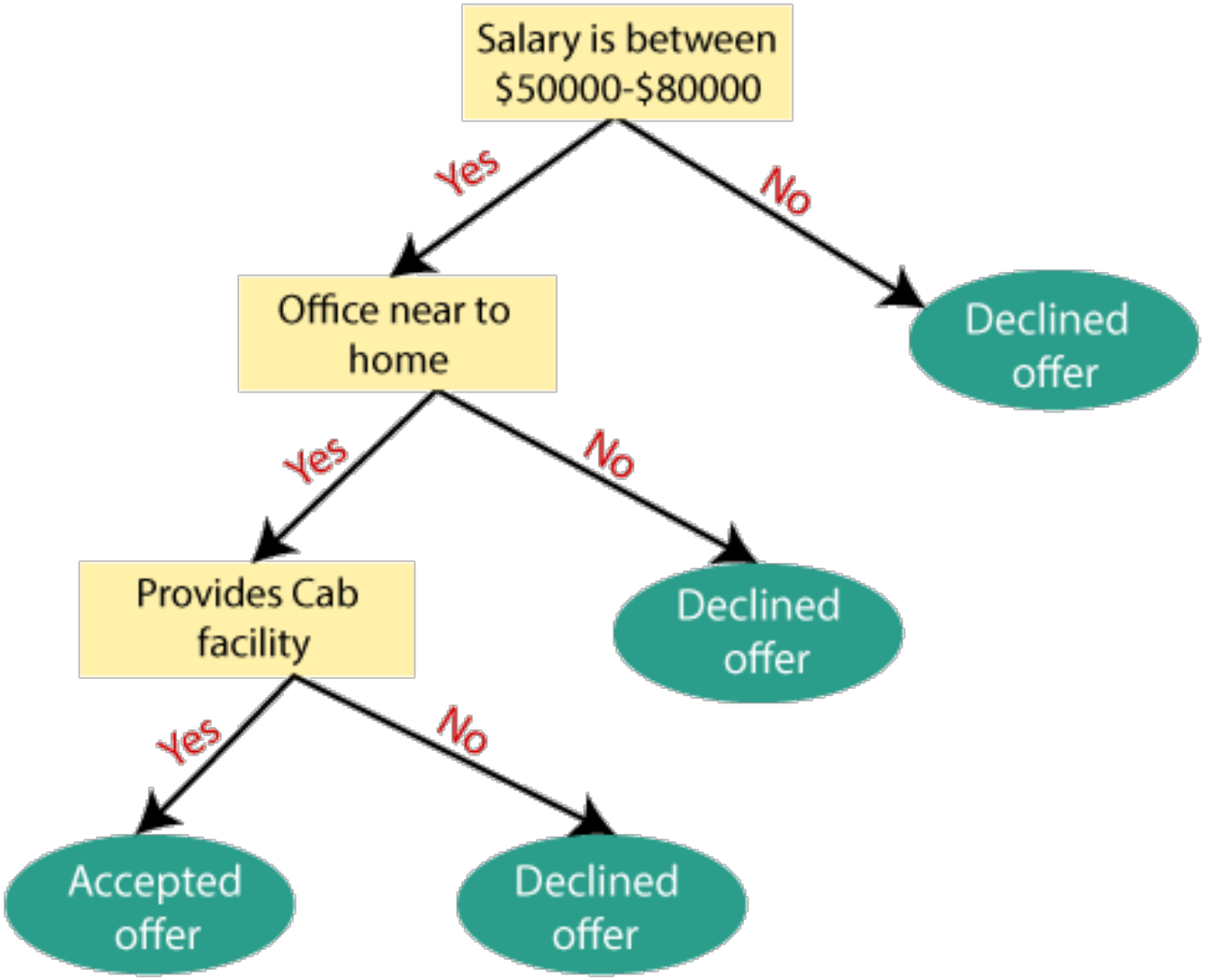
- For the next node, the algorithm again compares the attribute value with the other sub-nodes and move further. It continues the process until it reaches the leaf node of the tree. The complete process can be better understood using the algorithm.

Algorithm

- **Step-1:** Begin the tree with the root node, says S, which contains the complete dataset.
- **Step-2:** Find the best attribute in the dataset using **Attribute Selection Measure (ASM).**
- **Step-3:** Divide the S into subsets that contains possible values for the best attributes.
- **Step-4:** Generate the decision tree node, which contains the best attribute.
- **Step-5:** Recursively make new decision trees using the subsets of the dataset created in step -3. Continue this process until a stage is reached where you cannot further classify the nodes and called the final node as a leaf node.

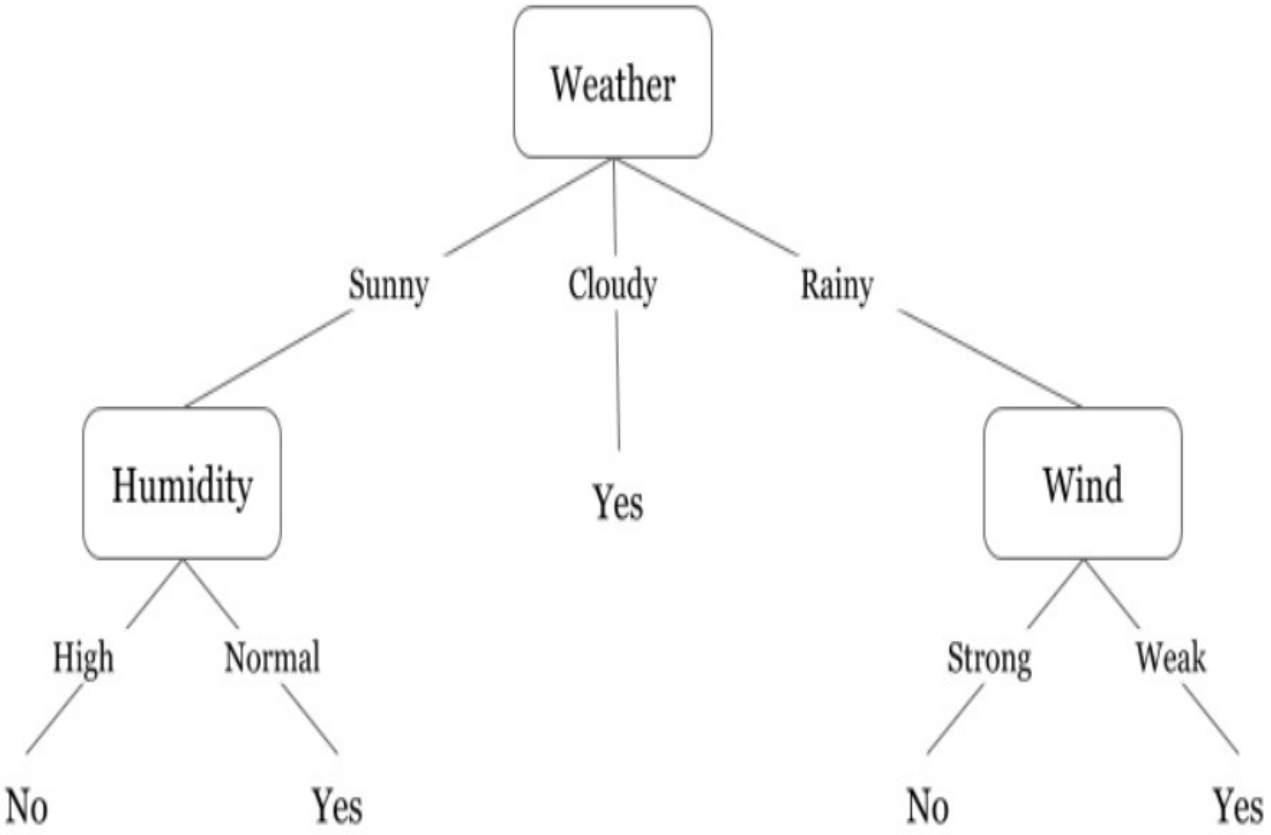
Example

- Suppose there is a candidate who has a job offer and wants to decide whether he should accept the offer or Not.
- So, to solve this problem, the decision tree starts with the root node (Salary attribute by ASM).
- The root node splits further into the next decision node (distance from the office) and one leaf node based on the corresponding labels.
- The next decision node further gets split into one decision node (Cab facility) and one leaf node.
- Finally, the decision node splits into two leaf nodes (Accepted offers and Declined offer).


Salary is between $50000-$80000
Yes
No
Office near to home
Declined offer
Yes
No
Provides Cab facility
Declined offer
Yes
No
Accepted offer
Declined offer

Example of a decision tree

Day	Weather	Temperature	Humidity	Wind	Play?
1	Sunny	Hot	High	Weak	No
2	Cloudy	Hot	High	Weak	Yes
3	Sunny	Mild	Normal	Strong	Yes
4	Cloudy	Mild	High	Strong	Yes
5	Rainy	Mild	High	Strong	No
6	Rainy	Cool	Normal	Strong	No
7	Rainy	Mild	High	Weak	Yes
8	Sunny	Hot	High	Strong	No
9	Cloudy	Hot	Normal	Weak	Yes
10	Rainy	Mild	High	Strong	No



We see that if the *weather is cloudy* then we must go to play. Why didn't it split more? Why did it stop there?
To answer this question, we need to know about few more concepts like entropy, information gain, and Gini index.

Attribute Selection Measures

- While implementing a Decision tree, the main issue arises that how to select the best attribute for the root node and for sub-nodes.

- So, to solve such problems there is a technique which is called as **Attribute selection measure or ASM.**

- By this measurement, we can easily select the best attribute for the nodes of the tree.

- There are two popular techniques for ASM, which are:

1. **Information Gain**
2. **Gini Index**

Information Gain

- Information gain is the measurement of changes in entropy after the segmentation of a dataset based on an attribute.
- It calculates how much information a feature provides us about a class.
- According to the value of information gain, we split the node and build the decision tree.
- A decision tree algorithm always tries to maximize the value of information gain, and a node/attribute having the highest information gain is split first.
- It can be calculated using the below formula:

Information Gain= Entropy(S)- [(Weighted Avg) *Entropy(each feature)]

- **Entropy:** Entropy is a metric to measure the impurity in a given attribute. It specifies randomness in data. Entropy can be calculated as:

$$\textbf{Entropy(s)= -P(yes)log2 P(yes)- P(no) log2 P(no)}$$

Where,

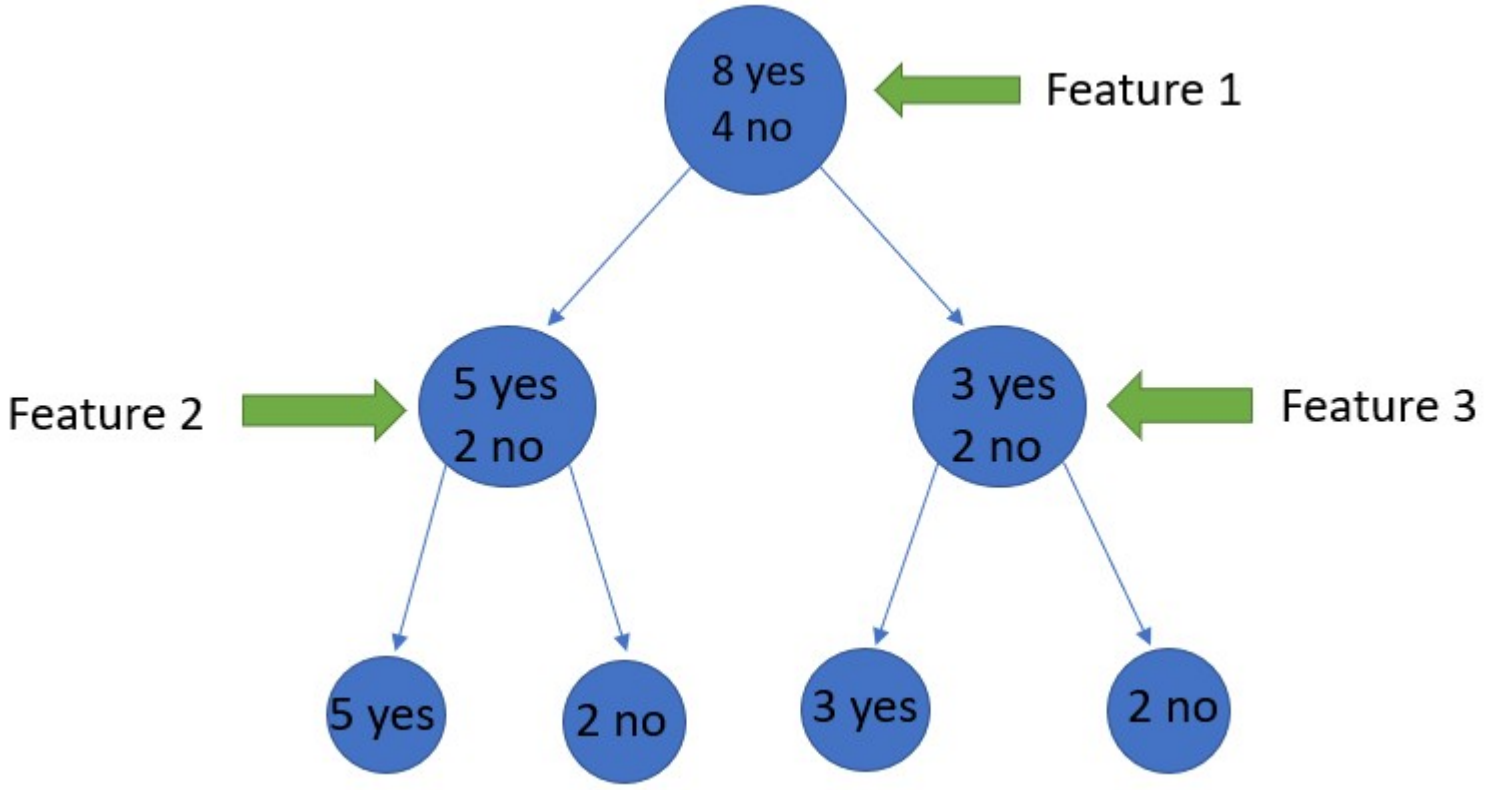
S= Total number of samples
P(yes)= probability of yes
P(no)= probability of no

How to calculate Entropy?

- Suppose a *feature* has 8 “yes” and 4 “no” initially, after the first split the left node *gets 5 ‘yes’ and 2 ‘no’* whereas right node *gets 3 ‘yes’ and 2 ‘no’*.

- We see here the split is not pure, why? Because we can still see some negative classes in both the nodes. In order to make a decision tree, we need to calculate the impurity of each split, and when the purity is 100%, we make it as a leaf node.

- To check the impurity of feature 2 and feature 3 we will take the help for Entropy formula.



We can clearly see from the tree itself that left node has low entropy or more purity than right node since left node has a greater number of “yes” and it is easy to decide here.

Always remember that the **higher the Entropy, the lower will be the purity and the higher will be the impurity.**

For feature 2 :

$$\Rightarrow \quad -\left(\frac{5}{7}\right)log_2\left(\frac{5}{7}\right) - \left(\frac{2}{7}\right)log_2\left(\frac{2}{7}\right)$$

$$\Rightarrow \quad -(0.71 * -0.49) - (0.28 * -1.83)$$

$$\Rightarrow \quad -(-0.34) - (-0.51)$$

$$\Rightarrow \quad 0.34 + 0.51$$

$$\Rightarrow \quad 0.85$$

For feature 3 :

$$\Rightarrow \quad -\left(\frac{3}{5}\right)log_2\left(\frac{3}{5}\right) - \left(\frac{2}{5}\right)log_2\left(\frac{2}{5}\right)$$

$$\Rightarrow \quad -(0.6 * -0.73) - (0.4 * -1.32)$$

$$\Rightarrow \quad -(-0.438) - (-0.528)$$

$$\Rightarrow \quad 0.438 + 0.528$$

$$\Rightarrow \quad 0.966$$

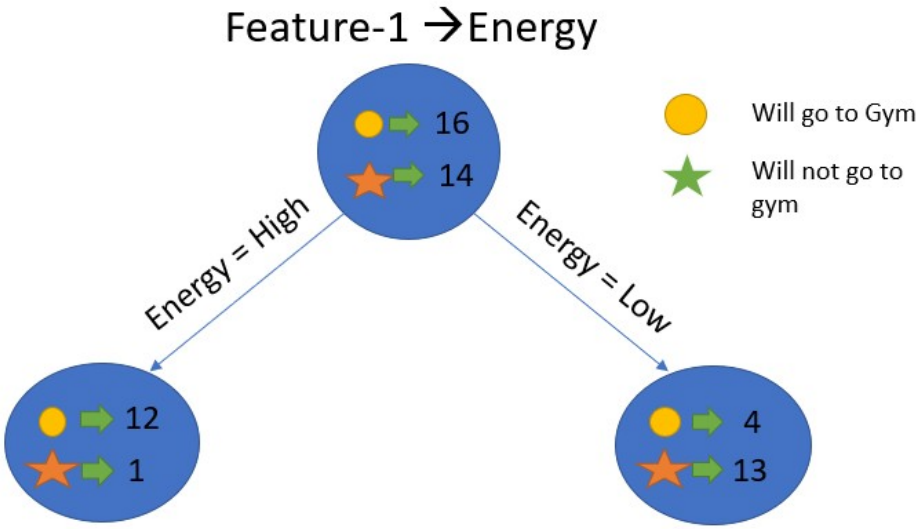
Example to calculate Information Gain

Suppose our entire population has a total of 30 instances. The dataset is to predict whether the person will go to the gym or not.

Let's say 16 people go to the gym and 14 people don't

Now we have two features to predict whether he/she will go to the gym or not.

- Feature 1 is **"Energy"** which takes two values *"high" and "low"*
- Feature 2 is **"Motivation"** which takes 3 values *"No motivation", "Neutral" and "Highly motivated".*



Let's calculate Entropy

$$E(Parent) = -\left(\frac{16}{30}\right)\log_2\left(\frac{16}{30}\right) - \left(\frac{14}{30}\right)\log_2\left(\frac{14}{30}\right) \approx 0.99$$

$$E(Parent|Energy = "high") = -\left(\frac{12}{13}\right)\log_2\left(\frac{12}{13}\right) - \left(\frac{1}{13}\right)\log_2\left(\frac{1}{13}\right) \approx 0.39$$

$$E(Parent|Energy = "low") = -\left(\frac{4}{17}\right)\log_2\left(\frac{4}{17}\right) - \left(\frac{13}{17}\right)\log_2\left(\frac{13}{17}\right) \approx 0.79$$

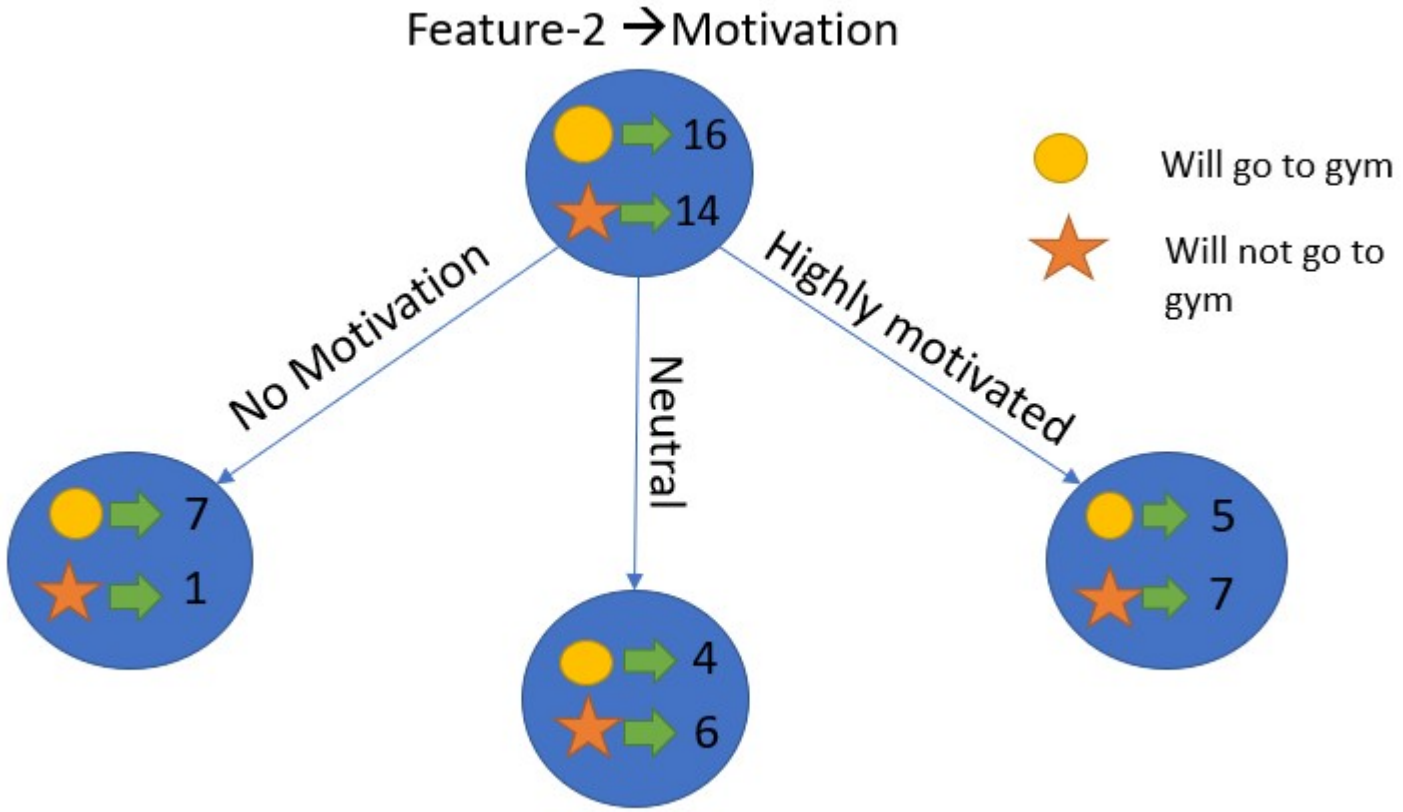
To see the weighted average of entropy of each node we will do as follows:

$$E(Parent|Energy) = \frac{13}{30} * 0.39 + \frac{17}{30} * 0.79 = 0.62$$

Now we have the value of E(Parent) and E(Parent|Energy), information gain will be:

$$\begin{aligned} Information\ Gain &= E(parent) - E(parent|energy) \\ &= 0.99 - 0.62 \\ &= 0.37 \end{aligned}$$

Similarly, we will do this with the other feature “Motivation” and calculate its information gain.



Let's calculate the entropy here:

$$E(Parent) = 0.99$$

$$E\Big(Parent|Motivation = "No\ motivation"\Big) = -\left(\frac{7}{8}\right)log_2\left(\frac{7}{8}\right) - \frac{1}{8}\log_2\left(\frac{1}{8}\right) = 0.54$$

$$E\big(Parent|Motivation = "Neutral"\big) = -\left(\frac{4}{10}\right)log_2\left(\frac{4}{10}\right) - (\frac{6}{10})\log_2\left(\frac{6}{10}\right) = 0.97$$

$$E\big(Parent|Motivation = "Highly\ motivated"\big) = -\left(\frac{5}{12}\right)log_2\left(\frac{5}{12}\right) - (\frac{7}{12})\log_2\left(\frac{7}{12}\right) = 0.98$$

To see the weighted average of entropy of each node we will do as follows:

$$E(Parent|Motivation) = \frac{8}{30} * 0.54 + \frac{10}{30} * 0.97 + \frac{12}{30} * 0.98 = 0.86$$

Now we have the value of E(Parent) and E(Parent|Motivation), information gain will be:

$$\begin{aligned} Information\ Gain &= E(Parent) - E(Parent|Motivation) \\ &= 0.99 - 0.86 \\ &= 0.13 \end{aligned}$$

We now see that the "Energy" feature gives more reduction which is 0.37 than the "Motivation" feature. Hence we will select the feature which has the highest information gain and then split the node based on that feature.

In this example "Energy" will be our root node and we'll do the same for sub-nodes. Here we can see that when the energy is "high" the entropy is low and hence we can say a person will definitely go to the gym if he has high energy, but what if the energy is low? We will again split the node based on the new feature which is "Motivation".

Gini Index

- Gini index is a measure of impurity or purity used while creating a decision tree in the CART(Classification and Regression Tree) algorithm.
- An attribute with the low Gini index should be preferred as compared to the high Gini index.
- It only creates binary splits, and the CART algorithm uses the Gini index to create binary splits.
- Gini index can be calculated using the below formula:

$$\textbf{Gini Index} = 1 - \sum_j P_j^2$$

When to stop splitting?

Usually, real-world datasets have a large number of features, which will result in a large number of splits, which in turn gives a huge tree. Such trees take time to build and can lead to overfitting. That means the tree will give very good accuracy on the training dataset but will give bad accuracy in test data. There are many ways to tackle this problem through hyperparameter tuning.

1. ***max_depth*** parameter - set the maximum depth of our decision tree. The more the value of ***max_depth***, the more complex your tree will be. Hence you need a value that will not overfit as well as underfit our data.

2. ***min_samples_split*** - set the minimum number of samples for each spilt.

3. ***min_samples_leaf*** – represents the minimum number of samples required to be in the leaf node. The more you increase the number, the more is the possibility of overfitting.

4. ***max_features*** – it helps us decide what number of features to consider when looking for the best split.

Pruning

- ***Pruning is a process of deleting the unnecessary nodes from a tree in order to get the optimal decision tree.***

- A too-large tree increases the risk of overfitting, and a small tree may not capture all the important features of the dataset.

- Therefore, a technique that decreases the size of the learning tree without reducing accuracy is known as Pruning.

- There are mainly two types of tree **pruning** technology used:

  Cost Complexity Pruning

  Reduced Error Pruning.

- There are mainly 2 ways for pruning:

  (i) **Pre-pruning** – we can stop growing the tree earlier, which means we can prune/remove/cut a node if it has low importance **while growing** the tree.

  (ii) **Post-pruning** – once our **tree is built to its**

Advantages of the Decision Tree

- It is simple to understand as it follows the same process which a human follow while making any decision in real-life.
- It can be very useful for solving decision-related problems.
- It helps to think about all the possible outcomes for a problem.
- There is less requirement of data cleaning compared to other algorithms.

Disadvantages of the Decision Tree

- The decision tree contains lots of layers, which makes it complex.
- It may have an overfitting issue, which can be resolved using the **Random Forest algorithm.**
- For more class labels, the computational complexity of the decision tree may increase.